MCU-Friendly Mamba

Optimizing Mamba Model Inference on Microcontrollers

Bartosz Drabiński[1]

Supervisor: Bo Yang[1]
Responsible Professor: Qing Wang[1]

[1]**EEMCS, Delft University of Technology, The Netherlands**

A Thesis Submitted to EEMCS Faculty Delft University of Technology,
In Partial Fulfilment of the Requirements
For the Bachelor of Computer Science and Engineering
May 31, 2026

Name of the student: Bartosz Drabiński
Final project course: CSE3000 Research Project
Thesis committee: dr. Qing Wang, Mark Neerincx

Abstract

Machine learning is becoming increasingly pervasive. While state-of-the-art models continue to grow in size and computational cost, Edge AI targets devices with strict limits on processing power and memory. TinyML — the branch of machine learning focused on running models on microcontrollers (MCUs) — exemplifies these constraints.

Meeting these constraints requires new model designs and careful adaptation of existing architectures. The Mamba architecture, built around State-Space Models (SSMs), is a promising candidate due to its compact parameterization and strong performance on long-context tasks. In this paper we survey recent efforts to deploy Mamba on microcontrollers and propose architecture and optimization strategies tailored to MCU constraints.

Keywords: TinyML, Mamba, microcontroller, Edge AI

1 Introduction

Machine learning has recently enabled solutions to problems that were difficult or infeasible with traditional algorithms. This progress produced both large, general-purpose models (e.g., large language models) and many compact, task-specific models.

Most deployments still rely on centralized data centres, which are efficient but impose limitations: they require network connectivity, add latency, and raise privacy concerns.

Edge AI reduces these limitations by running inference locally on user devices. TinyML focuses on the extreme end of this spectrum, designing models that run on microcontrollers (MCUs). MCUs are severely resource-constrained: typical devices provide one or two CPU cores (clocked below 240 MHz) and on the order of 512 KB of RAM, so they cannot host large models or their full-precision weights. Low latency is also essential because many TinyML applications (for example, audio or video streams) require real-time processing.

Numerous model families have been developed for different trade-offs, including multilayer perceptrons (MLPs), convolutional neural networks (CNNs), recurrent neural networks (RNNs), and Transformers. The recently proposed Mamba architecture [1] has emerged as a promising alternative to Transformers for long-context tasks. Mamba combines ideas from CNNs and RNNs and leverages Selective State-Space Model (S4) blocks to achieve a smaller memory footprint while maintaining strong performance on long sequences.

Prior work has explored optimizations for Mamba such as quantization [2], pruning, and knowledge distillation [3]. MambaLite-Micro [4] implemented PyTorch-style inference for Mamba directly in C and ran it on ESP32-S3 and STM32 microcontrollers; this preserves compatibility but makes architecture changes and further optimizations cumbersome.

This paper asks the following question: How can Mamba be adapted and optimized for efficient inference on microcontrollers? To answer it, we evaluate deployment strategies, identify memory and latency bottlenecks, and measure the impact of GPU-oriented optimizations when applied to MCUs.

Section 2 surveys related work and background. Section 4 describes the embedded deployment setup. Section **??** reports the experimental results. Section 6 discusses ethical considerations. Section 5.1 interprets the findings. Finally, Section 7 concludes and outlines future work.

2 Related Work and Background

2.1 TinyML

TinyML concerns the design, optimization, and deployment of machine learning models on highly resource-constrained embedded devices such as microcontrollers. The neural networks being run on them usually need to be trained with the constraints of the target device in mind, and then optimized for inference, as the typical device they need to run on have less than 256 KB of DRAM.

Those constraints also usually limit the architectures that can be used in a viable way. For example the most common choices are convolutional neural networks (CNNs) and recurrent neural networks (RNNs), which can be more parameter-efficient than Multi-Layer Perceptrons (MLPs) [5].

The computational power of MCUs is also limited, so latency is a critical concern. Many TinyML applications (for example, audio or video streams) require real-time processing. This sometimes makes using models based on non-trivial mathematical operations difficult to deploy. This can be especially important for activation layers using trigonometric functions (such as Tanh), or even exponentiation (such as Softmax or SiLU) [6].

The lack of quick floating point support on many MCUs also makes quantization a common optimization strategy. It works by reducing the precision of the model's weights and activations from 32-bit floats to formats like 8, or even 4-bit integers. This can significantly reduce the model size and memory requirements. This improvement comes at the cost of potential accuracy loss, and the memory gains are not always proportional because of the overhead [7]. Moreover, in some cases,quantization can lead to significant accuracy degradation if not done carefully, especially for models that are not designed with quantization in mind [2].

Besides quantization, other common optimization strategies include pruning, knowledge distillation and low-rank factorization have been proposed [8]. Pruning removes redundant parameters from the model, while knowledge distillation transfers knowledge from a larger, more accurate model to a smaller one. Those techniques can be more complex to implement, but often come with possible size reductions of up to 50% [3].

2.2 Mamba architecture

The Mamba architecture [1] is a recently new architecture proposition. Its main innovation is combining Convolutional Neural Network with Recurrent Neural Network ideas. This design allows Mamba to capture long-range dependencies while maintaining a smaller memory footprint than Transformers.

Its main building block is the Selective State-Space Model (S6) layer, which is intended to capture long-range dependencies in sequences while avoiding the quadratic complexity of attention. This layer is explicidly designed to break the standard Linear Time-Invariant (LTI) bottleneck of RNNs, which limits their ability to capture long-range dependencies. By using SSMs, Mamba can achieve strong performance on long-context tasks while maintaining a smaller memory footprint than Transformers. It is based around the standard SSM formulation:

$$h_t = \overline{A}h_{t-1} + \overline{B}x_t$$
$$y_t = Ch_t + Dx_t$$

However then the SSM matrices are computed based on the input. This allows Mamba to adapt its dynamics based on the input sequence, which can be interpreted as the model weighting different parts of the input differently at each time step.

It has also been designed with hardware efficiency in mind. The SSM layers can be implemented using efficient scan (parallel-prefix) algorithms, which can be parallelized on modern hardware, because of its linear nature. However, this SSM-based architecture poses a big challenge, because the reccurent approach makes it sensitive to quantization [2; 9].

Most of the research has been focused on making the models smaller, but only to the point of making them fit on a mobile device. For example, Physics-Guided Tiny-Mamba Transformer [10] is brought down to around 0.5 million parameters and run on a Jetson Nano; Tiny-ViM [11] creates a 5 million parameter model for vision tasks; FEMBA [9] while it introduces aggresive quantization, it still uses 7.8 million parameters.These models, while implement techniques to reduce the size, are still too large to be deployed on microcontrollers.

There have also been efforts to create models that are small enough to be deployed on microcontrollers, such as Light-Mamba [12], which uses SSMs for Ultra Wideband indoor positioning. However, it has not been deployed on an actual microcontroller.

Little research has been done on deploying Mamba on microntrollers. Notably MambaLite-Micro [4] implemented PyTorch-style inference for Mamba directly in C and ran it on ESP32-S3 and STM32 microcontrollers. This approach preserves compatibility but makes architecture changes and further optimizations cumbersome. It has also argued that the current tooling for MCU deployment is not yet mature.

2.3 Research gap

Two primary gaps motivate this work. First, many techniques for optimizing TinyML models have been developed, but their applicability to Mamba is not well understood. Second, while MambaLite-Micro demonstrates feasibility, it does not explore architecture modifications or optimizations that could further reduce memory usage and latency on MCUs. The deployment pipeline it develops is also difficult to modify, because the inference process is tightly coupled with the model architecture. This paper aims to fill these gaps by systematically evaluating deployment strategies, identifying bottlenecks specific to Mamba on microcontrollers and proposing improvements to the discovered limitations.

3 Proposed Mamba-MCU Optimizations

3.1 Embedded Deployment

Deployment of machine learning models on microcontrollers while possible still poses some issues. MambaLite-Micro authors decided to pursue a path that would allow them to maintain exact same inference results as the trained PyTorch model, by extracting the weights from the model and hard-coding it in pure C. This approach, however, comes with many drawbacks, mainly that making any modifications to the model would need to also be implemented in C.

For this paper we decided not to go that route, and instead choose an already existing tool for porting a model to a microcontroller. There have been many different frameworks created, notably TensorFlow Lite Micro (now being renamed to LiteRT) [13] and ExecuTorch [14]. We have investigated both of them and found that the hardware agnostic workflow of TFLM gives us more freedom for experimentation, has better documentation and causes less issues for deployment. This approach allows us to avoid many intermediary forms of the neural network (notably we do not convert the model to ONNX at any point). This eliminates the conversion differences between the interpretations from being a factor in the experiments.

Additionally there have been many tools built into and around TFLM. We are using the tflm_model_transforms, which strips all redundant and debug data from the model, which allows for fair comparison between model sizes by reducing the model graph overhead. We also use AI Edge Quantizer, which allows for precise control over the quantization process, and it operates directly on the tflite model files.

We also looked into other deployment options before deciding on the final one. The most interesting ones are Brevitas [15] (also used by FEMBA) and Torchao [16]. They both allow for quantization to be done on the PyTorch form of the model, and both of them allow for Quantization Aware Training, with the Brevitas additionally allowing for bit width quantization down to 1. In the final implementation we decided to not use them because of the need for kernel fusion that would arise after the QAT, which would be difficult to achieve efficiently for microcontrollers.

Because of time constraints of this project we also implemented and tested the models only on the ESP32 (Figure 1) family of microcontrollers (ESP32, and ESP32S3). This was done using esp-tflite-micro, which is a vendor specific implementation of the TFLM. We decided against using a vendor specific deployment pipeline, such as ESP-DL or STM32Cube.AI, to make the results not be bound to only one MCU platform.

3.2 Model Architecture

As in this work we do not focus on investigating the hyperparameters of the Mamba architecture, we use the same archi-

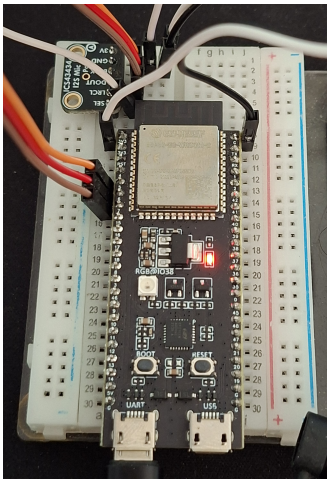

Figure 1: Photo of an ESP32S3 with ICS43434 microphone module attached

tecture as the MambaLite-Micro [4]. This allows us to have a close comparison to the previous work, and to focus on the deployment and optimization strategies.

The architecture consists of a fully connected input layer, a single Mamba block, average pooling layer, and a fully connected output layer. The Mamba block input is of size 64, and the expansion ratio is 2.

As deployment of the original Mamba implementation is not possible for TFLM, because of the custom GPU kernels it uses, we re-implemented the architecture in PyTorch. This allows us to have a more flexible deployment pipeline, and to be able to make changes to the architecture and optimizations more easily. We do this by altering the activation functions, as well as reducing the amount of exponentiation operations used.

3.3 Model Architecture

As in this work we do not focus on investigating the hyperparameters of the Mamba architecture, we use the same architecture as the MambaLite-Micro [4]. This allows us to have a close comparison to the previous work, and to focus on the deployment and optimization strategies.

The architecture consists of a fully connected input layer, a single Mamba block, average pooling layer, and a fully connected output layer. The Mamba block input is of size 64, and the expansion ratio is 2.

As deployment of the original Mamba implementation is not possible for TFLM, because of the custom GPU kernels it uses, we re-implemented the architecture in PyTorch. This allows us to have a more flexible deployment pipeline, and to be able to make changes to the architecture and optimizations more easily. We do this by altering the activation functions, as well as reducing the amount of exponentiation operations used.

3.4 Deployment Optimization

For the deployment optimization, we focus on quantization, as it is one of the most common and effective optimization strategies for TinyML applications. We evaluate INT8 weights and activations quantization.

Because of the lack of any looping mechanism in TFLM, the SSM implementation is unrolled by default. This causes the model size to grow linearly with the sequence length, which is an issue for the larger sequence length of the Speech Commands dataset. To mitigate this, we split the deployment model into 3 parts - pre-ssm, ssm step, and post-ssm. Then inside of the inference code we loop over the ssm step, which allows us to reuse the same weights for each time step and significantly reduce the model size. We also evaluate the impact of this on the HAR dataset, where both implementations are possible, to understand the trade-offs between the two approaches.

4 Experimental Setup and Evaluation

4.1 Dataset and Preprocessing

There exist many different datasets that have been used and demosntrated to be viable for TinyML applications. They are usually based on some kind of sensor data, such as audio, video or accelerometer data. For this work I decided to use two datasets: the Human Activity Recognition (HAR) dataset [17] and the Google Speech Commands dataset [18].

They are chosen because of their popularity in the TinyML community, and because they represent two different types of data (accelerometer and audio). It also allows us to create some kind of comparison to the previous work, especially the MambaLite-Micro [4] which also uses those datasets.

The HAR dataset consists of recordings of volunteers performing activities of daily living while carrying a waist-mounted smartphone with embedded inertial sensors. The data is already heavily preprocessed into 561 features. For the Mamba architecture following the previous work, I pad this input to 570 features, and split them into 10 time steps of 57 features each. State-of-the-art models can achieve nearly 98% accuracy on this dataset [19].

For the Google Speech Commands dataset, we preprocess the audio data into Mel-frequency cepstral coefficients (MFCCs), which are commonly used features for audio classification tasks. We extract 40 MFCC features for every time step, which results in 51 time steps with 40 features each. While some implementations choose to reduce the number of classed from the original 35 to 10, I decided to keep all 35 classes, because the Mamba architecture is designed to handle long sequences, and it allows us to test the limits of the architecture in a more challenging setting. State-of-the-art models can achieve around 96% accuracy on this dataset [20].

4.2 Experimental Design

For the experimental design, it is important to evaluate the impact of different parts of the deployment pipeline. This includes the model architecture, the optimization strategies, and the deployment choices. It is also important to measure both memory usage and latency, as they are both critical for TinyML applications.

The main goal is to be able to measure the peak memory usage and latency of the model during inference on a microcontroller. This can be challenging for an embedded device,

but TFML provides tools for this. For memory usage, we can use the TFML Memory Recorder. It allows us to measure the peak memory of the model itself, as well as the split between permamently allocated and temporary memory. For latency, we created a modified version of the default TFML profiler. It allows us to measure the total time taken for inference, as well as the time taken for each operation type. This allows us to identify bottlenecks in the inference process and understand how different optimizations impact latency.

For testing accuracy we run the model on the entire test set of each dataset while the model is on the PC where the training and optimization is done. We do this both with the PyTorch implementation, as well as after the conversion to TFML, to make sure that the conversion process does not introduce any significant accuracy degradation. The accuracy is also measured after the model is quantized, to check what is the impact of quantization on the model's performance.

We also test accuracy of the model on the microcontroller. Because of the limited resources of the microcontroller, we cannot run the entire test set on it. Instead, we run a subset of 50 samples from the test set to ensure that the accuracy is not compromised by the final C++ implementation.

5 Results and Discussion

This section presents the experimental results of the impact of different techniques on the accuracy and memory usage. Results should be supported by descriptive and clear figures.

Effect of Quantization on Memory and Accuracy
This subsection presents:

- Memory reductions across quantization strategies
- Accuracy changes induced by quantization
- Variance of performance across repeated runs

Embedded Efficiency Trade-offs
This subsection presents:

- Inference latency measurements
- Model size comparisons
- Accuracy-efficiency relationships

Effects of other memory optimizations on accuracy and latency
This subsection presents:

- Impacts of Low rank factorization

5.1 Discussion

This section interprets the experimental findings and discusses their implications for embedded trustworthy AI systems. It also discusses the limitations of the approach.

Trade offs between accuracy, memory efficiency, and latency
Discuss:

- How making models smaller reduces accuracy of them
- How some optimizations can be made that hurt latency

How Mamba model performs
Discuss:

- The performance of Mamba compared to SOTA for the dataset

Limitations
Discuss:

- Dataset limitations
- More quantization possible
- Hardware specific implementation
- Difficulty of comparison with other models
- Lack of standardized tooling

6 Responsible Research

6.1 Ethical Considerations

While this work does not involve third party participants, it's also important to consider its impacts. The main focus of this work is to allow running more efficient machine learning on microcontrollers. Spreading the TinyML ideas can help move some of the computations from datacenters and powerful devices to low-powered ones. This in turn helps the inevitable transition towards wide spread usage of ML be more sustainable.

On the other hand being able to run video and audio classification could be used for malicious purposes such as invading someone's privacy.

6.2 Reproducibility

The datasets used in this research are publicly available. All of the software used in this research had licenses allowing for use in research and has been properly acknowledged.

All the steps needed to replicate the findings from this paper have also documented. This includes the initial model training, conversion and quantization processes, as well as the way the model was benchmarked on the microcontroller. While full replication also requires the specific MCU, the one chosen for this research is widely available, and seen all over the TinyML field.

We put utmost care to ensure the metrics are accurate and can be compared between each other and are relevant when comparing them to other works. The same profiler has been used for all experimental runs, and all results have been annotated with which version of the MCU was used to gather them.

The entire code for the project is publicly available. It also contains instructions on how to run it, and what libraries versions have been used to allow for easy replication setup creation.

6.3 Use of AI Tools

Artificial intelligence tools were used as supportive instruments during the development of this work. Their use was limited to writing parts of the code, assisting with debugging, and refining written text for clarity and grammatical correctness.

Specifically, AI assistance was used for:

- Writing scripts to setup the experiment pipeline
- Debugging implementation issues in experimental code
- Improving clarity and structure of written explanations
- Assisting with formatting and consistency in LaTeX writing

All scientific decisions, experimental design choices, and interpretation of results were made independently by the author. AI tools were not used to generate or alter experimental outcomes, results, or conclusions.

7 Conclusions and Future Work

This section should directly answer the research question and summarize the main findings.

The conclusion should:

- Restate the research objective
- Summarize the most important findings
- Explain the memory limitations of Mamba models on MCUs
- Highlight key trade-offs identified in the experiments

Future work may discuss:

- Other memory optimization strategies
- More standardized deployment (LiteRT custom operation)
- Energy-aware optimization

The conclusion should remain concise and readable as a standalone section.

References

[1] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces."

[2] Z. Xu, Y. Yue, X. Hu, Z. Yuan, Z. Jiang, Z. Chen, J. Yu, C. Xu, S. Zhou, and D. Yang, "MambaQuant: Quantizing the Mamba Family with Variance Aligned Rotation Methods."

[3] I. F. Shihab, S. Akter, and A. Sharma, "Efficient Unstructured Pruning of Mamba State-Space Models for Resource-Constrained Environments," *arXiv preprint*, 2025.

[4] H. Xu, J. Xia, W. Yang, Y. Sui, and S. Xia, "MambaLite-Micro: Memory-Optimized Mamba Inference on MCUs."

[5] M. Tri Lê, P. Wolinski, and J. Arbel, "Efficient Neural Networks for Tiny Machine Learning: A Comprehensive Review," vol. 17, no. 4, pp. 1–41.

[6] T. Cassimon, S. Vanneste, S. Bosmans, S. Mercelis, and P. Hellinckx, "Designing resource-constrained neural networks using neural architecture search targeting embedded devices," vol. 12, p. 100234.

[7] K. Dokic, M. Martinovic, and D. Mandusic, "Inference speed and quantisation of neural networks with TensorFlow Lite for Microcontrollers framework," in *2020 5th South-East Europe Design Automation, Computer Engineering, Computer Networks and Social Media Conference (SEEDA-CECNSM)*, pp. 1–6.

[8] T. Suwannaphong, F. Jovan, I. Craddock, and R. McConville, "Optimising TinyML with quantization and distillation of transformer and mamba models for indoor localisation on edge devices," vol. 15, no. 1, p. 10081.

[9] A. Tegon, N. Lehmann, Y. Li, A. Cossettini, L. Benini, and T. M. Ingolfsson, "FEMBA on the Edge: Physiologically-Aware Pre-Training, Quantization, and Deployment of a Bidirectional Mamba EEG Foundation Model on an Ultra-low Power Microcontroller."

[10] C. Li, D. Huang, K. Yao, X. Ni, L. Shen, and F. Luo, "Physics-Guided Tiny-Mamba Transformer for Reliability-Aware Early Fault Warning."

[11] X. Ma, Z. Ni, and X. Chen, "TinyViM: Frequency Decoupling for Tiny Hybrid Vision Mamba."

[12] T. Wang, Y. Shu, L. Qiao, D. Ding, and G. Li, "LightMamba: A Resource-Efficient Deep-Learning Method for UWB NLOS Identification Based on Selective State-Space Modeling," vol. 13, no. 5, pp. 8735–8748.

[13] R. David, J. Duke, A. Jain, V. J. Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regev, R. Rhodes, T. Wang, and P. Warden, "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems."

[14] M. Nachin, D. Desai, S. S. Jia, C. Lai, M. Liu, J. Szwejbka, R. Alvarez, R. Ascani, D. Bort, M. Candales, *et al.*, "ExecuTorch - a unified PyTorch solution to run AI models on-device," *arXiv preprint arXiv:2605.08195*, 2026.

[15] G. Franco, A. Pappalardo, and N. J. Fraser, "Xilinx/brevitas," 2025.

[16] A. Or, A. Jain, D. Vega-Myhre, J. Cai, C. D. Hernandez, Z. Zheng, D. Guessous, V. Kuznetsov, C. Puhrsch, M. Saroufim, S. Rao, T. Tran, and A. Samardžić, "TorchAO: PyTorch-Native Training-to-Serving Model Optimization."

[17] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "A Public Domain Dataset for Human Activity Recognition Using Smartphones,"

[18] P. Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition," *ArXiv e-prints*, Apr. 2018.

[19] Y. Enokibori, "rTsfNet: A DNN model with Multi-head 3D Rotation and Time Series Feature Extraction for IMU-based Human Activity Recognition."

[20] J. Wang, L. Yu, L. Huang, C. Zhou, H. Zhang, Z. Song, Z. Ma, and Z. Zhang, "Efficient Speech Command Recognition Leveraging Spiking Neural Network and Curriculum Learning-based Knowledge Distillation,"